Deep Learning (AI2100/AI5100/CS5480)
Assignment–1

Deadline: 9th March, 2026

Submission: (`A1_<roll-number>.zip`) **Total: 200 Marks**

Please read the following instructions before starting the assignment:

- It is **strongly recommended** that you work on this homework on an individual basis. If you have any questions or concerns, feel free to contact the instructor or the TAs at least 2 days before deadline.
- Use numpy for basic functions like log, sqrt, power. Do not use other built-in functions and implimentation should be from scratch.
- You can also use markdown cells in the notebook to explain theoretical problems and observations.
- Some questions have specific submission requirements as mentioned in the respective problem statements. Please carefully follow those guidelines (e.g., submission of `.py` files, `.pdf` reports, plots, etc.).
- Ensure that all submitted code runs without errors. Non-executable code will receive zero marks for the corresponding question.
- Include a `README.md` file clearly describing the folder structure of the submitted zip file, indicating where the solution for each question is located, along with instructions to run the code.

---

1 Artificial Neural Network with Backpropagation (20 Marks)

a) Implement a simple Artificial Neural Network (ANN) from scratch (i.e., without using built-in functions). Implement the back-propagation algorithm to learn the weights of an ANN with 2 input nodes, 2 hidden nodes and 1 output node. Train your network to learn the following functions:

   i) XOR (10)
   ii) Cosine (10)

   Choose the activation functions and loss functions appropriately. The number of nodes in the network should be a configurable parameter in your code. Generate the dataset by perturbing the input binary digits with a small amount of Gaussian noise. Use 80% of the dataset for training and the remaining for testing. Report the following:

   i) Training and testing loss over training iterations. Plot these values.
   ii) Training and testing accuracy over training iterations. Plot these values.
   iii) Print the training and test loss and accuracy at the end of training.
   iv) Deterministic Gradient Descent: Experiment with the number of training samples n and see how it affects performance. In other words, repeat the training and testing for three different choices of n.
   v) Stochastic Gradient Descent: Experiment with the batch size m (for a fixed n) and see how it affects performance. In other words, repeat the training and testing for three different choices of m.

2 Universal Approximation Theorem (15 marks)

(a) State a formal version of the Universal Approximation Theorem for a feedforward neural network with a single hidden layer and a sigmoid (or any non-polynomial continuous) activation function. Clearly specify: **(4)**

- the class of target functions being approximated,
- assumptions on the domain,
- the notion of approximation and the norm used.

(b) Outline the main idea behind a proof of the Universal Approximation Theorem. A full formal proof is *not* required. Your answer should briefly explain the intuition or the key theoretical argument (e.g., constructive approximation using basis-like functions, or a connection to classical results from approximation theory such as the Stone–Weierstrass theorem). **(4)**

(c) Discuss **two** important limitations of the Universal Approximation Theorem from a practical machine learning perspective. For each limitation, explain why it is relevant when designing or training neural networks. **(4)**

(d) Briefly comment on how the Universal Approximation property changes when using ReLU activations and allowing network depth to increase while keeping the width relatively small. **(3)**

3 One Step of Gradient Descent in a Deep Neural Network (25 marks)

Consider the following feedforward neural network with three affine layers (four layers including the input), ReLU activations in the hidden layers, and two output classes.

Network architecture

- Input: $x = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \in \mathbb{R}^2$
- **Layer 1 (Affine + ReLU):**

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(1)} = \mathrm{ReLU}(z^{(1)})$$

$$W^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

- **Layer 2 (Affine + ReLU):**

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \quad a^{(2)} = \mathrm{ReLU}(z^{(2)})$$

$$W^{(2)} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad b^{(2)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

- **Layer 3 (Output layer, affine only):**

$$s = W^{(3)}a^{(2)} + b^{(3)}$$

$$W^{(3)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad b^{(3)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

There are two classes, and the true class label is $y = 1$ (using 1-based indexing). The learning rate is $\eta = 0.1$.

Assume the derivative of the ReLU function is

$$\mathrm{ReLU}'(t) = \begin{cases} 1, & t > 0 \\ 0, & t \leq 0 \end{cases}$$

Part A: Softmax with Cross-Entropy Loss **(12 Marks)**

The softmax function is defined as

$$q_i = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

and the cross-entropy loss for a single example is

$$L_{\text{CE}} = -\log q_y$$

(a) Perform the forward pass and compute $z^{(1)}, a^{(1)}, z^{(2)}, a^{(2)}, s$, and the softmax probabilities q. **(3)**

(b) Compute the cross-entropy loss L_{CE}. **(1)**

(c) Compute the gradient of the loss with respect to the logits s. **(1)**

(d) Using backpropagation, compute the gradients with respect to all weights and biases:

$$\nabla_{W^{(3)}}, \nabla_{b^{(3)}}, \nabla_{W^{(2)}}, \nabla_{b^{(2)}}, \nabla_{W^{(1)}}, \nabla_{b^{(1)}}$$

(4)

(e) Perform one gradient descent update step for all parameters and report the updated values. **(3)**

Part B: Multiclass Hinge Loss **(13 marks)**

The multiclass hinge loss is defined as

$$L_{\text{hinge}} = \sum_{j \neq y} \max\left(0, s_j - s_y + 1\right)$$

(a) Compute the hinge loss for the given example. **(2)**

(b) Compute the gradient of the hinge loss with respect to the logits s. **(2)**

(c) Using backpropagation, compute the gradients with respect to all weights and biases. **(6)**

(d) Perform one gradient descent update step for all parameters and report the updated values. **(3)**

4 A Voyage into Neural Networks

This assignment takes you on a journey through the historical development of neural networks, from the simple perceptron (1958) to ideas in modern deep learning. You will implement each model from scratch, understand why each advancement was necessary, and empirically demonstrate the limitations and capabilities of each approach. Side by side, this will also require you to create *your own* dataset. Instructions for both parts will be specified incrementally.

Dataset: IIT-H Campus Life Prediction

You will create and progressively augment a tabular dataset based on student life at IIT-H. The dataset will be used to predict **mess duration** (how long a student would spend in the mess for a particular meal).

Dataset Construction

Collect/simulate data with 500 samples collected over 15 days containing:

- `student_ID` (integer)
- `day_of_week` (0-6, Monday=0)
- `meal_time` (0: breakfast, 1: lunch, 2: dinner)

- `meal_type` (0: vegetarian, 1: non-vegetarian)
- `is_weekend` (binary)
- `class_day` (0: if class is there that day, 1: otherwise)
- `assignment_deadline` (0: if there is an assignment deadline that day, 1: otherwise)
- `weather_features` (temperature (°C), humidity, wind_speed, rain, air_quality)
- `rising_time` (minutes from 00:00:00)
- `sleeping_time` (minutes from 00:00:00)
- `mess_duration` (**target**; minutes)

Then extend this dataset to 5000 samples by adding Gaussian noise to the `weather_features`, `rising_time`, and `sleeping_time` features.

Submission Requirement: Include a `data_generation.py` script and `dataset_description.pdf` explaining your data collection/generation methodology.

5 ADALINE (Adaptive Linear Neuron) (1960)

5.1 Theory (20 points)

a) Discuss the ADALINE work introduced by Widrow & Hoff (1960) and provide the ADALINE learning algorithm.

b) Show that ADALINE has a strong convergence guarantee to the global least-squares solution, and is independent of data separability.

5.2 Implementation (10 points)

Implement the ADALINE with the following specifications:

```python
class Adaline:
    def __init__(self, learning_rate=1.0, max_iterations=1000):
        """
        Initialize the perceptron
        learning_rate: step size for weight updates
        max_iterations: maximum training iterations
        """
        pass

    def fit(self, X, y):
        """
        Train the perceptron
        X: (n_samples, n_features) array
        y: (n_samples,)
        Returns: training history (mistakes per epoch)
        """
        pass

    def predict(self, X):
        """Return predicted outputs"""
        pass

    def score(self, X, y):
        """Return mean square error"""
        pass
```

5.3 Experiments (10 points)

Using the dataset you have collected:

a) Plot the dataset;

b) Train Adaline and plot:

- Mean-Squared error vs. epoch
- Decision boundary in 2D (project to 2 principal components)
- Training and validation error vs. epoch

c) Experiment with learning rates $\eta \in \{0.01, 0.1, 1.0, 10.0\}$. Plot convergence curves and discuss the effect.

d) Vary the amount of training data from 10% to 100% in increments of 10%. Plot final accuracy vs. training set size.

Deliverables: Code (`adaline.py`), plots, and analysis in report.

6 Multi-Layer Perceptron with Backpropagation

6.1 Theory (10 points)

a) Explain the vanishing gradient problem. Show mathematically why it occurs with sigmoid activation in deep networks.

b) Instead of a fully connected feedforward network, if we were to consider a random graph model where hidden units were arbitrarily dropped with probability 0.5, would applying the chain rule change? If so, how?

6.2 Implementation (15 points)

Implement a flexible MLP framework:

```python
class MLP:
    def __init__(self, layer_sizes, activations,
                 loss='cross_entropy', learning_rate=0.01,
                 optimizer='sgd', batch_size=32,
                 weight_init='xavier', regularization=None,
                 lambda_reg=0.01):
        """
        layer_sizes: list of layer dimensions [input, hidden1, ..., output]
        activations: list of activation functions for each layer
        loss: 'mse' or 'cross_entropy'
        optimizer: 'sgd', 'momentum', 'adam'
        weight_init: 'random', 'xavier', 'he'
        regularization: None, 'l1', 'l2'
        """
        pass

    def forward(self, X):
        """Forward pass, return activations at each layer"""
        pass

    def backward(self, X, y):
        """Backward pass, compute gradients"""
        pass

```

```python
    def fit(self, X_train, y_train, X_val, y_val, epochs=100):
        """
        Train with early stopping based on validation loss
        Returns: training history
        """
        pass
```

Implement the following components separately for modularity:

- Activation functions (sigmoid, tanh, ReLU, leaky ReLU) and their derivatives
- Loss functions (MSE) and their derivatives
- Optimizers (SGD, momentum, Adam, Nesterov-Accelerated GD, AdaGrad, RMSProp, Muon)
- Weight initialization schemes (Xavier-Glorot, He)

6.3 Experiments: Architecture Ablation (15 points)

Using the dataset:

6.3.1 Depth Ablation

a) Fix hidden layer width at 64 neurons and vary depth:

- 1 hidden layer: [input, 64, output]
- 2 hidden layers: [input, 64, 64, output]
- 3 hidden layers: [input, 64, 64, 64, output]
- 4 hidden layers: [input, 64, 64, 64, 64, output]

b) For each architecture:

- Plot train/validation loss and accuracy curves
- Report final test accuracy
- Analyze training time per epoch
- Check for overfitting (train-val gap)

c) Create a summary plot: Test accuracy vs. depth

6.3.2 Width Ablation

1. Fix depth at 2 hidden layers and vary width in the follow ranges:
   (a) [input dimension, 8, 8, output dimension]
   (b) [input dimension, 16, 16, output dimension]
   (c) [input dimension, 32, 32, output dimension]
   (d) [input dimension, 64, 64, output dimension]
   (e) [input dimension, 128, 128, output dimension]
   (f) [input dimension, 256, 256, output dimension]
2. Plot test accuracy vs. number of parameters (log scale)
3. Discuss the width vs. depth performance relationship

6.3.3 Activation Function Comparison

a) Fix architecture at [input, 64, 64, output] and compare:

- Sigmoid in all layers
- Tanh in all layers
- ReLU in hidden layers, sigmoid in output
- Leaky ReLU in hidden layers, sigmoid in output

b) For each configuration:

- Plot loss curves
- Report gradient statistics (mean, std) during training
- Analyze dead neurons (for ReLU variants)

c) Verify vanishing gradients experimentally: measure gradient magnitude at each layer for sigmoid vs. ReLU networks

6.4 Loss Function Analysis (10 points)

a) Come up 3 different loss functions to solve the problem and compare the results obtained using these 3 different loss functions. Discuss which loss is more suitable and why.

6.5 Optimizer Comparison (10 points)

Using the best architecture from above:

a) Use the following PyTorch optimizers and compare:

- SGD
- SGD with momentum ($\beta = 0.9$)
- Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
- AdaGrad
- Nesterov accelerated GD
- RMSProp
- Muon

b) For each optimizer:

- Plot convergence curves (use same learning rate initially, then tune separately)
- Report time to reach 90% of best validation accuracy
- Analyze update directions (plot weight update magnitudes)

c) Learning rate sensitivity: for each optimizer, plot final accuracy vs. learning rate

6.6 Regularization (5 points)

a) Add L1 and L2 regularization to your MLP

b) Compare:

- No regularization
- L1 with $\lambda \in \{0.001, 0.01, 0.1\}$
- L2 with $\lambda \in \{0.001, 0.01, 0.1\}$

c) Plot train vs. validation accuracy for each

d) For L1: plot weight sparsity (fraction of weights $< 10^{-3}$) vs. λ

6.7 Success Analysis (5 points)

a) Visualize learned features: extract hidden layer activations and project to 2D using PCA. Color by class label.

b) Compare MLP performance vs. Adaline. Create a comparison table.

Deliverables: Code (`mlp.py`, `activations.py`, `optimizers.py`), extensive plots, and detailed analysis.

7 Neural Network Features for Kernel Methods (Advanced)

This section explores using neural networks as feature extractors for kernel-based methods.

7.1 Theory (10 points)

a) Review kernel methods: Define a valid kernel function. State Mercer's theorem.

b) Explain how a neural network's hidden layer can be viewed as a learned feature representation $\phi(x)$. How does this relate to the kernel trick?

7.2 Implementation (20 points)

7.2.1 Neural Network as Feature Extractor

a) Consider the best MLP that you have learnt so far

b) Extract features from the penultimate layer (before output): $\phi_{\text{NN}}(x) = h^{(L-1)}(x)$

c) Visualize NN features using t-SNE plot

d) Implement a kernel SVM using these neural features:

```python
# Define kernel using NN features
def neural_kernel(x_i, x_j, feature_extractor):
    """
    k(x_i, x_j) = <phi_NN(x_i), phi_NN(x_j)>
    """
    phi_i = feature_extractor(x_i)
    phi_j = feature_extractor(x_j)
    return np.dot(phi_i, phi_j)
```

e) Train kernel Support Vector Regression (SVR) with neural kernel on Phase 4 data

7.2.2 Comparison with Standard Kernels

Implement and compare the following kernels for SVR:

a) Linear kernel: $k(x_i, x_j) = x_i^T x_j$

b) Polynomial kernel: $k(x_i, x_j) = (x_i^T x_j + c)^d$ (try $d \in \{2, 3\}$)

c) RBF (Gaussian) kernel: $k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ (try $\gamma \in \{0.01, 0.1, 1.0\}$)

d) Neural kernel: $k(x_i, x_j) = \phi_{\text{NN}}(x_i)^T \phi_{\text{NN}}(x_j)$

For each kernel:

- Report test accuracy
- Plot decision boundaries (project to 2D)
- Compute and visualize the kernel matrix $K_{ij} = k(x_i, x_j)$